Optimized Projectiles

Overview **2**
Setting up the asset **3**
Demo scene **4**
VFX Prefab Structure **5**
One projectile consists of 3 different vfx prefabs: **5**
Launch Flash and Hit **5**
Projectile 5
Optimizations and Emission Control **6**
Memory Optimization 6
Render Optimization 6
Emission control per particle System 6
Projectile System **7**
Further Optimization Options **8**
Support & Contact **9**

Overview

A modern projectile VFX package focused on precision, performance, and scalability. Every effect is built to reduce memory usage and batch calls by using texture atlases, shared prefabs, and a minimal material setup, making it ideal for both desktop and mobile projects.

A custom shader setup with vertex streams allows emission to be controlled per particle system instead of per material. This gives you more flexibility without increasing draw calls or duplicating materials.

Note

This ***documentation is unified across all projectile assets*** in this series.
Setup, demo scene controls, and prefab structure are consistent between all packages, ensuring a familiar workflow and quick onboarding.

Please note that screenshots may differ between assets.
Some packages include meshes, while others are intentionally meshless to allow for improved mobile performance and easier optimization.

The included Projectile System and Particle System Utils modules may be updated or extended in newer projectile assets over time. I aim to keep these modules up to date across all packages, including older releases. However, it is possible that an older asset may still contain a previous module version with missing features or utility scripts.
If you notice any discrepancies or missing functionality when comparing your local asset modules with the online documentation, please let me know. The online documentation always reflects the most up to date module versions, and I will update the asset accordingly to match it.

Setting up the asset

This asset uses URP as default.
Make sure that ShaderGraph is installed in your project.

If you are using BIRP or HDRP please use the upgrade packages to reimport the render pipeline specific resources.
Navigate to SF Studio → SF Projectiles Vol1 → Upgrades and import the resources by double clicking on the pipeline specific unitypackage.

HDRP

When using HDRP, projectiles may appear *less bright* because HDRP supports significantly higher emission values compared to the other pipelines.

This asset includes the Particle System Utils helper module, which provides several tools for batch editing particle systems.

You can find the module and its documentation under:
SF Studio → Shared → Modules → Particle System Utils

To adjust emission intensity, it is recommended to use the ParticleSystemEmissionMultiplier tool. This allows you to quickly increase or decrease emission values across multiple particle systems.

To use the tool, navigate to:
Tools → SF Studio → Particle System Utils → Emission Multiplier Editor

Then follow these steps:
Drag and drop all relevant particle system prefabs into the editor.
Enter a multiplier value for the emission strength.
Apply the multiplier to update all selected particle systems.

Demo scene

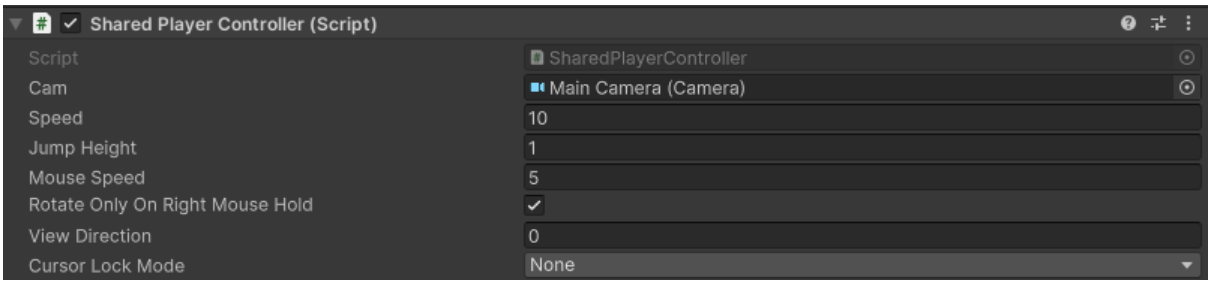
The demo scene comes with an integrated PlayerController

Movement Controls:

- W – Move forward
- A – Move left
- S – Move backward
- D – Move right
- Space – Jump

Camera Rotation:

- Default Mode:
  - Hold the right mouse button to rotate the camera.
- Free Rotation Mode:
  - If you prefer to rotate the camera directly with mouse movement (without holding the right mouse button), simply disable the RotateOnlyOnRightMouse flag on the Player GameObject in your scene.



Shoot Projectiles:

- Left Mouse button – shoot current projectile
- 1 – select previous projectile
- 2– select next projectile

VFX Prefab Structure

One projectile consists of 3 different vfx prefabs:

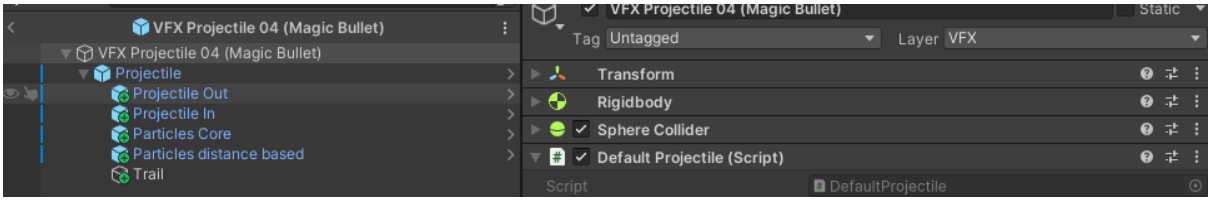
- Projectile
- Launch Flash
- Hit VFX

Launch Flash and Hit

The launch flash and hit effects are implemented as pure Particle System prefabs.
The root GameObject is an empty particle system, allowing the effect to be played directly.
All child objects represent the individual visual effects that make up the prefab.

Projectile

The projectile root GameObject contains all functional components such as the collider, rigidbody, and the DefaultProjectile script. The visual effects are attached as a “Projectile” child object. This object includes an empty particle system that allows the entire effect to be played at once. All remaining child objects represent the individual visual elements that make up the prefab.



Optimizations and Emission Control

This asset includes several optimizations designed to reduce memory usage, improve rendering performance, and allow full control over particle emissions.

Memory Optimization

Every GameObject that contains a particle system is created as a variant of the root particle system prefab. This approach keeps prefab sizes small and avoids unnecessary duplication.

Render Optimization

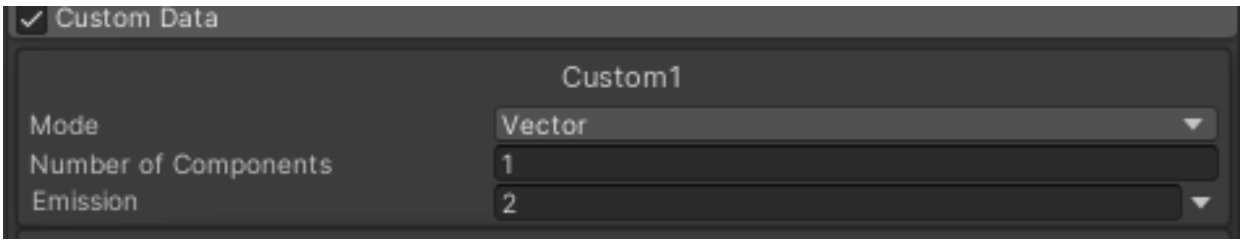
To minimize draw calls and improve rendering performance, the included effects use texture atlases wherever possible, reducing the total number of materials.
Additionally, individual particle systems use the Order in Layer setting in the Renderer section to group effects that share the same materials into the same batches.

Emission control per particle System

The projectiles use the Custom Vertex Stream particle shaders from the included ParticleShaders module. This allows emission behavior to be controlled directly from within the particle system.

The root particle system prefab already has the custom vertex stream enabled, and all particle variants based on this prefab inherit the same setup.

To adjust the emission strength, open a projectile prefab and navigate to the Custom Data section of the particle system. Tweaking the X value of the Custom1 data increases or decreases the particle emission intensity.



Projectile System

This asset includes a simple projectile system module that allows you to shoot projectiles. The system automatically handles the spawning of launch flash effects and hit effects.
All included modules can be found under: *SF Studio → Shared → Modules*

Each projectile has the Default Projectile script attached. This script requires both a Sphere Collider and a Rigidbody component to function correctly.

The demo scene also includes a Projectile Spawner script. This allows you to select from a list of available projectiles and spawn them by pressing the left mouse button.

For a **detailed overview** of all configuration options and advanced settings, please refer to the **dedicated module documentation** located at:
SF Studio → Shared → Modules → ProjectileSystem

Further Optimization Options

The included projectiles use individual Order In Layer values in the Particle System Renderer for each material.

If your purchased projectile VFX asset includes meshes, the Particle System module provides additional options to further optimize batch calls (**Sorting Layer Editor**)

All meshes already use dedicated Order In Layer values, for example one value for additive unlit materials and another for alpha blended unlit materials. However, Unity cannot efficiently batch different meshes when they share the same sorting layer, even if they use identical materials.

To further reduce batch calls, you can ensure that identical meshes using the same materials are rendered in a single batch across multiple projectiles. Simply open the particle system that renders a mesh and assign a dedicated Sorting Layer ID in the Renderer settings of that particle system. This allows Unity to group matching meshes more efficiently and improves overall render performance.



You can repeat this setup for each mesh type and set a dedicated layer for different meshes. E.g. bullet mesh to Bullet sorting layer, circle mesh to Circle sorting layer, etc.

This approach allows Unity to batch draw calls more efficiently and can noticeably improve performance when many projectiles are active at the same time.

For your convenience There are some helper scripts in the included Particle System Utils module, which you can find under *SF Studio → Shared → Modules → Particle System Utils*

For a detailed overview of helper scripts and usages, please refer to the **dedicated module documentation**.

An alternative option is to manually adjust the Order in Layer value for each mesh. For example, you could use values like -4 and -5 for one projectile mesh, -6 and -7 for another, and so on. However, this method quickly becomes hard to manage, as you need to remember the order values for every individual mesh. Using dedicated sorting layer IDs is generally the cleaner and more maintainable solution.

Support & Contact

Thank you for using this asset!

If you encounter any issues, have questions, or want to suggest improvements, don’t hesitate to reach out: ***gamedevibk@gmail.com***

Your feedback is always welcome and helps improve both this asset and future updates

If you’re happy with the asset, I’d really appreciate it if you could take a moment to leave a short **review** on the Unity Asset Store. It truly helps a lot with visibility and supports further development.